

A* (A-Star) Search Algorithm

1. Aim

To implement the **A* (A-Star) search algorithm** to find the shortest and most efficient path between a given starting node and a goal node using the actual path cost and heuristic estimated cost.

2. Problem Statement

Finding the shortest path between a starting point and a destination is an important problem in Artificial Intelligence. The A* algorithm is used to determine the most efficient path by considering the cost already travelled and the estimated cost required to reach the destination. The algorithm should find the optimal path while avoiding unnecessary exploration of other possible paths.

3. Solution

The problem is solved using the **A* search algorithm**, which evaluates each node using the function $f(n) = g(n) + h(n)$. Here, $g(n)$ represents the actual cost from the starting node to the current node, while $h(n)$ represents the estimated cost from the current node to the goal. By selecting the node with the lowest $f(n)$ value, the algorithm efficiently finds the shortest path.

4. Algorithm

1. Start from the given starting node.
2. Add the starting node to the OPEN list.
3. Calculate $f(n) = g(n) + h(n)$ for the available nodes.
4. Select the node with the lowest $f(n)$ value.
5. Move the selected node to the CLOSED list.
6. Check all neighbouring nodes.
7. Calculate the path cost for each neighbouring node.
8. Update the cost and parent node if a better path is found.
9. Repeat the process until the goal node is reached.
10. Trace the parent nodes to obtain the shortest path.
11. Display the shortest path and its total cost.

5. Program

```
import heapq
```

```
def a_star(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))

    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            path = []

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()
            return path, g_cost[goal]

        for neighbor, cost in graph[current]:
            new_cost = g_cost[current] + cost

            if neighbor not in g_cost or new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                f_cost = new_cost + heuristic[neighbor]
```

```
        heapq.heappush(open_list, (f_cost, neighbor))
        parent[neighbor] = current

    return None, float("inf")
```

```
# Graph
```

```
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('E', 1)],
    'D': [('F', 3)],
    'E': [('F', 2)],
    'F': []
}
```

```
# Heuristic values
```

```
heuristic = {
    'A': 6,
    'B': 5,
    'C': 3,
    'D': 3,
    'E': 2,
    'F': 0
}
```

```
start = 'A'
```

```
goal = 'F'
```

```
path, cost = a_star(graph, heuristic, start, goal)
```

```
print("Shortest Path:", " -> ".join(path))
```

```
print("Total Cost:", cost)
```

6. Result

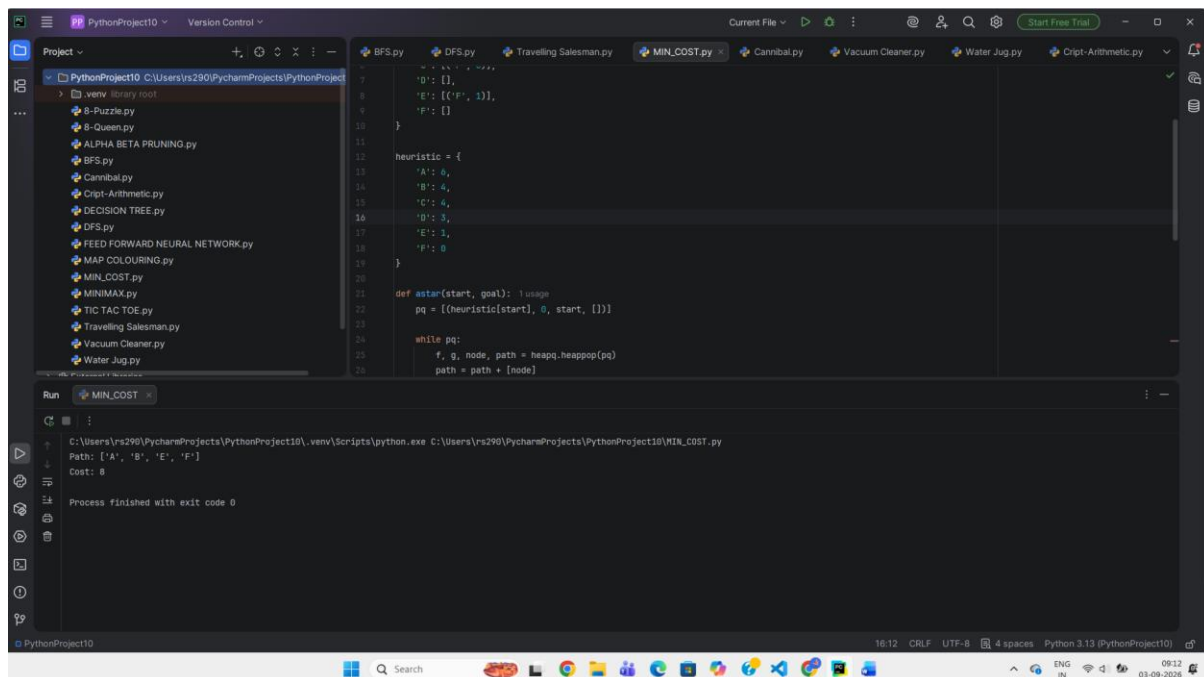
The A* search algorithm was successfully implemented in Python. It evaluated the possible paths using the actual cost and heuristic value and found the shortest path from node **A** to node **F**.

Output:

Shortest Path: A -> B -> D -> F

Total Cost: 6

Therefore, the A* algorithm successfully determined the **optimal path from the starting node A to the goal node F** with a total cost of **6**.



The screenshot displays a Python IDE with a project named 'PythonProject10'. The file explorer on the left shows various Python files, including 'MIN_COST.py'. The main editor window shows the implementation of the A* search algorithm. The code defines a graph, a heuristic function, and the A* search function. The output window at the bottom shows the execution results: 'Path: ['A', 'B', 'D', 'F']' and 'Cost: 6'. The process finished with exit code 0.

```
graph = {
    'A': ['B', 1],
    'B': ['C', 4],
    'C': ['D', 4],
    'D': ['E', 3],
    'E': ['F', 1],
    'F': []
}

heuristic = {
    'A': 6,
    'B': 4,
    'C': 4,
    'D': 3,
    'E': 1,
    'F': 0
}

def a_star(start, goal):
    pq = [(heuristic[start], 0, start, [])]
    while pq:
        f, g, node, path = heapq.heappop(pq)
        path = path + [node]
```

Run MIN_COST.py

C:\Users\rs290\PycharmProjects\PythonProject10\.venv\scripts\python.exe C:\Users\rs290\PycharmProjects\PythonProject10\MIN_COST.py

Path: ['A', 'B', 'D', 'F']

Cost: 6

Process finished with exit code 0